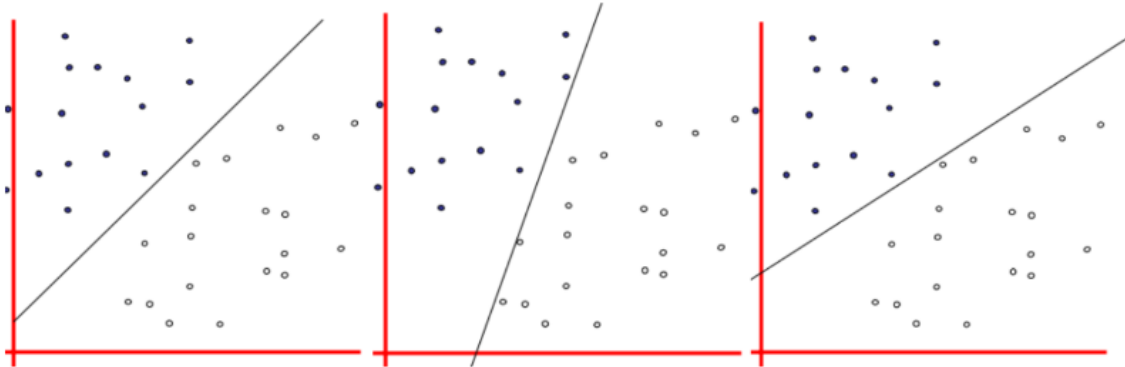


Support Vector Machine (SVM)

Algoritmo de **classificação** que cria um hiperplano com base nos vetores de suporte para dividir classes.



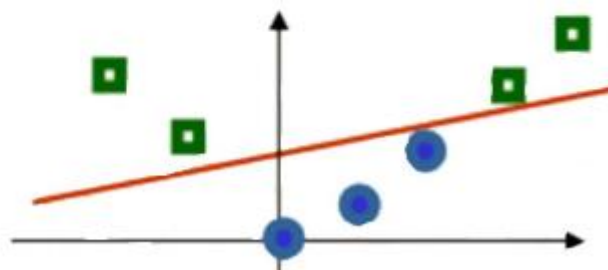
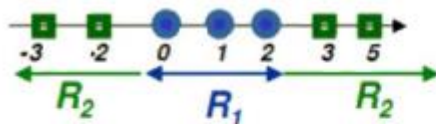
Fórmula

Não Relevante.

Hiperparâmetros

Kernel Trick (Transformação dos Dados)

Usado quando os dados não podem ser separados por uma linha reta.



- Transforma os dados para outra dimensão.
- Principais tipos de kernel:
 1. Kernel Linear
 2. Kernel Polinomial

3. Kernel Gaussiano (RBF/Radial)

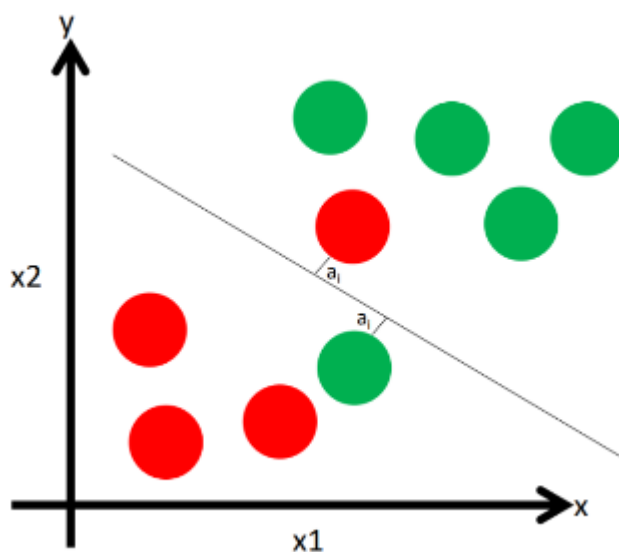
Característica	Linear	Polinomial	Gaussiano (RBF)
Hiperparâmetros Principais	C	C, grau d, γ, coef0 r	C, γ
Risco de Overfitting	Baixo	Médio a Alto	Alto
Quando Usar	Dados aproximadamente lineares	Relações não-lineares moderadas	Estrutura muito não-linear ou desconhecida
Custo Computacional	Baixo	Médio/Alto	Médio

C (Penalização dos Erros)

Controla o quão rígido o modelo é com erros.

$$\frac{1}{2} |w|^2 + c \sum_i a_i$$

- Penaliza classificações incorretas.
- C alto → tenta separar completamente, maior risco de overfitting.
- C baixo → permite mais erros, modelo mais simples.



Código Base

1. Importar e criar o modelo SVM

- Usar SVC() da biblioteca scikit-learn.
- Definir o kernel (linear, polinomial ou RBF) e o parâmetro C para penalização de erros.

2. Treinar o modelo

- Usar o método fit(X_train, y_train) para ensinar o SVM a separar as classes com base nos dados de treinamento.

3. Fazer previsões

- Aplicar predict(X_test) nos dados de teste para gerar as classificações do modelo.

4. Avaliar o desempenho

- Utilizar métricas como acurácia, matriz de confusão, precisão, recall ou F1-score para verificar como o modelo está separando as classes.

```
svm = SVC(kernel='rbf', C=2)

svm.fit(X_train, y_train)

prev = svm.predict(X_test)
```

```
# Instalar yellowbrick se necessário
# pip install yellowbrick

# Importar bibliotecas
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from yellowbrick.classifier import ConfusionMatrix
import matplotlib.pyplot as plt
```

```
# Carregar dataset de exemplo
iris = load_iris()
X = iris.data
y = iris.target

# Dividir em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Criar o modelo SVM
svm = SVC(kernel='rbf', C=1.0)

# Treinar o modelo
svm.fit(X_train, y_train)

# Fazer previsões
prevision = svm.predict(X_test)

# Avaliar desempenho com Yellowbrick
plt.figure(figsize=(5,5))
cm = ConfusionMatrix(svm, classes=iris.target_names) # Cria visualização
cm.fit(X_train, y_train) # Treina a visualização com dados de treino
cm.score(X_test, y_test) # Avalia o modelo e atualiza a visualização
cm.show() # Exibe a matriz de confusão

# Avaliar acurácia numericamente
print("Acurácia:", accuracy_score(y_test, prevision))

# Relatório detalhado
print("\nRelatório de Classificação:")
print(classification_report(y_test, prevision, target_names=iris.target_names))
```

Métricas de Erro: Matriz de Confusão

A matriz de confusão, no contexto de SVM, é uma tabela que mostra como o modelo classificou corretamente ou incorretamente cada classe, comparando as previsões do SVM com os valores reais.

```
cm = ConfusionMatrix(svm)

cm.fit(X_train, y_train)

cm.score(X_test, y_test)
```

Métricas de Desempenho

```
accuracy_score(y_test, prev)
```

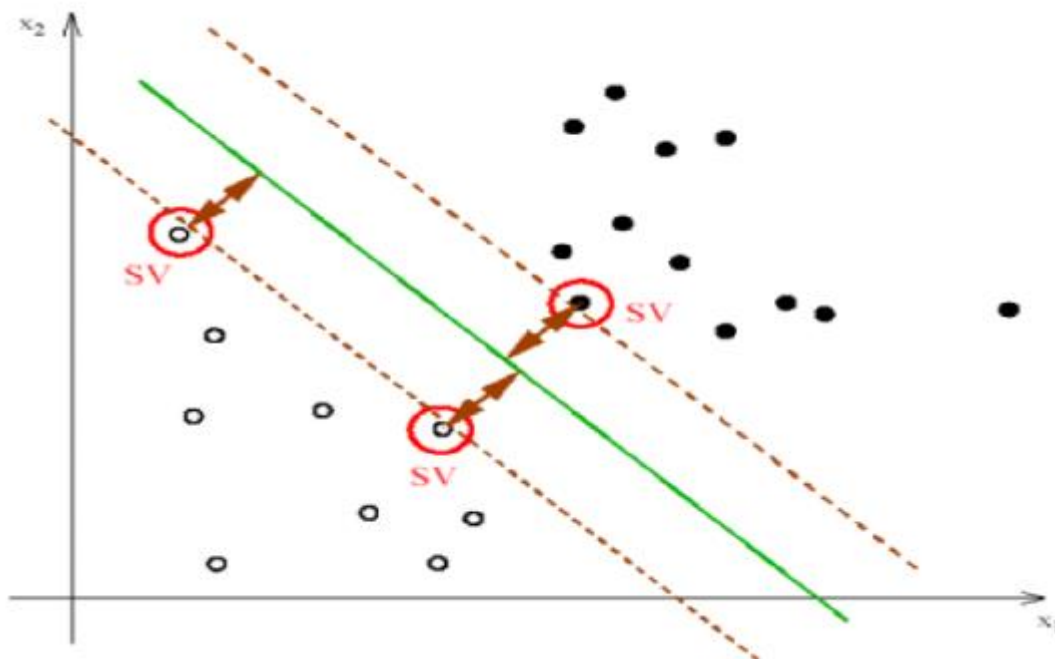
Resumo

- Procura forma de separar grandes grupos de dados.
 - Tenta encontrar linha (ou plano) que divide os grupos.
 - Tenta ao máximo MAXIMIZAR a margem e cria uma “zona de segurança”.
 - É potente para dados de alta dimensão, mas exige normalização e não escala bem para datasets gigantes.
-

Vetores de Suporte (Support Vectors)

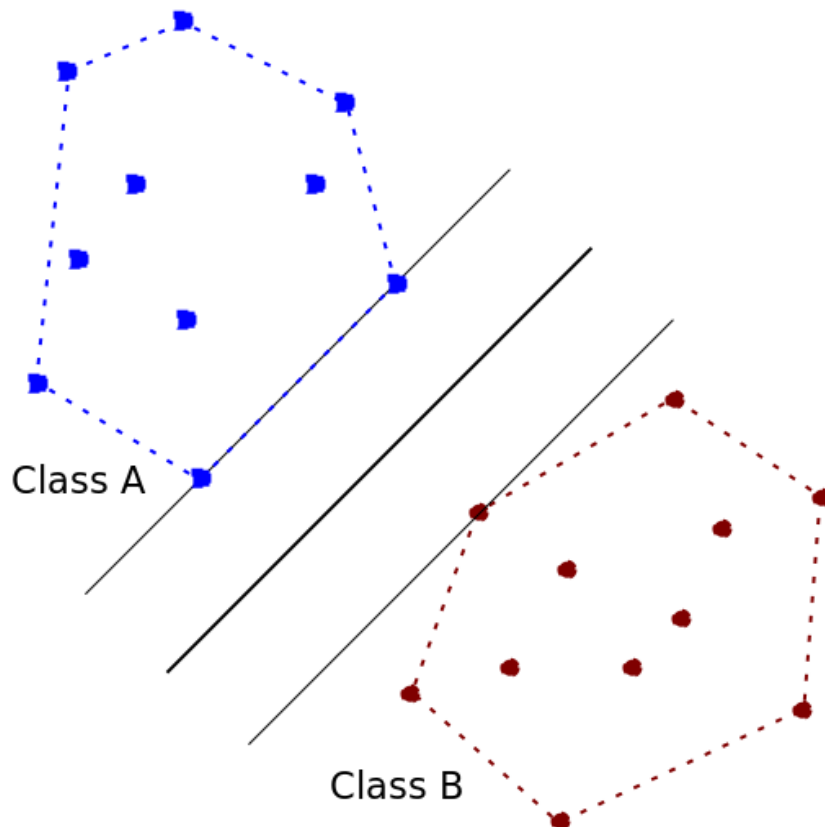
São os pontos mais próximos (um de cada classe) do hiperplano.

- Determinam a separação.
- O hiperplano só se move se um desses pontos for movido.



Convex Hull

Técnica que cria uma “envoltória” geométrica ao redor dos grupos.



- Ignora pontos internos, usa apenas as bordas.
- O hiperplano é colocado entre as duas envoltórias convexas.
- Os pontos na borda são os vetores de suporte.